

ООП (больше о классах)

Скрытый указатель «this» и цепочка функций-элементов

Один из вопросов о классах, который часто задают начинающие программисты: «Когда вызывается функция-элемент, как C++ отслеживает, к какому объекту она была вызвана?».

Для начала давайте определим простой класс для работы с ним. Этот класс инкапсулирует целочисленное значение и предоставляет несколько функций доступа для получения и установки этого значения:

```
#include <iostream>

class Simple
{
private:
    int m_id{};

public:
    Simple(int id)
        : m_id{ id }
    {
    }

    int getID() const { return m_id; }
    void setID(int id) { m_id = id; }

    void print() const { std::cout << m_id; }
};

int main()
{
    Simple simple{1};
    simple.setID(2);

    simple.print();

    return 0;
}
```

Как и следовало ожидать, эта программа выдает результат:

```
2
```

Каким-то образом, когда мы вызываем `simple.setID(2);`, C++ знает, что функция `setID()` должна работать с объектом `simple`, и что `m_id` на самом деле относится к `simple.m_id`.

Ответ заключается в том, что C++ использует скрытый указатель под названием `this`! В этом уроке мы рассмотрим `this` более подробно.

Скрытый `this` указатель

Внутри каждой функции-элемента ключевое слово `this` является константным указателем, который содержит адрес текущего неявного объекта.

В большинстве случаев мы не используем `this` явно, но просто чтобы доказать, что можем:

```
#include <iostream>

class Simple
{
private:
    int m_id{};

public:
    Simple(int id)
```

```

        : m_id{ id }
    {
    }

    int getID() const { return m_id; }
    void setID(int id) { m_id = id; }

    void print() const { std::cout << this->m_id; } // use `this` pointer to access the
implicit object and operator-> to select member m_id
};

int main()
{
    Simple simple{ 1 };
    simple.setID(2);

    simple.print();

    return 0;
}

```

Это работает аналогично предыдущему примеру и выводит:

2

Обратите внимание, что функции-элементы print() из двух предыдущих примеров делают одно и то же:

```

void print() const { std::cout << m_id; }          // implicit use of this
void print() const { std::cout << this->m_id; }    // explicit use of this

```

Оказывается, первое является сокращением для второго. Когда мы компилируем наши программы, компилятор неявно добавляет к любому элементу, ссылающемуся на неявный объект, this->. Это помогает сделать наш код более лаконичным и избежать необходимости многократно писать this-> вручную.

Напоминание

Мы используем -> для выбора элемента из указателя на объект. this->m_id эквивалентно (*this).m_id.

Как это this устанавливается?

Давайте подробнее рассмотрим вызов этой функции:

```
simple.setID(2);
```

Хотя кажется, что у вызова функции setID(2) есть только один аргумент, на самом деле их два! При компиляции компилятор переписывает выражение simple.setID(2); следующим образом:

```
Simple::setID(&simple, 2); // note that simple has been changed from an object prefix
to a function argument!
```

Обратите внимание, что теперь это просто стандартный вызов функции, а объект simple (который раньше был префиксом объекта) теперь передаётся по адресу в качестве аргумента функции.

Но это только половина ответа. Поскольку в вызове функции теперь есть дополнительный аргумент, определение функции-элемента также необходимо изменить, чтобы принимать (и использовать) этот аргумент в качестве параметра. Вот наше исходное определение функции-элемента для setID():

```
void setID(int id) { m_id = id; }
```

То, как компилятор переписывает функции, зависит от реализации, но конечный результат выглядит примерно так:

```
static void setID(Simple* const this, int id) { this->m_id = id; }
```

Обратите внимание, что наша функция `setID` имеет новый крайний левый параметр с именем `this`, который является константным указателем (это означает, что его нельзя перенаправить, но содержимое указателя можно изменить). Элемент `m_id` также был переписан как `this->m_id`, с использованием указателя `this`.

Для продвинутых читателей

В данном контексте ключевое слово `static` означает, что функция не связана с объектами класса, а рассматривается как обычная функция в области видимости класса. Мы рассмотрим статические функции-элементы в уроке [Статические функции-элементы](#).

Собирая все это воедино:

1. Когда мы вызываем `simple.setID(2)`, компилятор фактически вызывает `Simple::setID(&simple, 2)`, и `simple` передаётся функции по адресу.
2. Функция имеет скрытый параметр с именем `this`, который получает адрес `simple`.
3. Переменные-элементы внутри `setID()` имеют префикс `this->`, который указывает на `simple`. Поэтому, когда компилятор вычисляет `this->m_id`, он фактически вычисляет `simple.m_id`.

Хорошая новость в том, что всё это происходит автоматически, и не так важно, помните вы, как это работает, или нет. Всё, что вам нужно помнить, — это то, что все нестатические функции-элементы имеют `this` указатель, который ссылается на объект, для которого была вызвана функция.

Ключевое понимание

Все нестатические функции-элементы имеют указатель `this const`, который содержит адрес неявного объекта.

[this всегда указывает на объект, с которым выполняется операция](#)

Начинающие программисты иногда не понимают, сколько существует `this` указателей. У каждой функции-элемента есть один `this` параметр-указатель, который указывает на неявный объект. Рассмотрим:

```
int main()
{
    Simple a{1}; // this = &a inside the Simple constructor
    Simple b{2}; // this = &b inside the Simple constructor
    a.setID(3); // this = &a inside member function setID()
    b.setID(4); // this = &b inside member function setID()

    return 0;
}
```

Обратите внимание, что указатель `this` попеременно содержит адрес объекта `a` или `b` в зависимости от того, вызвали ли мы функцию-элемент объекта `a` или `b`.

Поскольку `this` — это просто параметр функции (а не элемент), он не увеличивает объём памяти, занимаемый экземплярами вашего класса.

[Явная ссылка this](#)

В большинстве случаев вам не нужно будет явно ссылаться на указатель `this`. Однако в некоторых случаях это может быть полезно:

Во-первых, если у вас есть функция-элемент, у которой есть параметр с тем же именем, что и у элемента данных, вы можете устранить неоднозначность, используя `this`:

```
struct Something
{
    int data{}; // not using m_ prefix because this is a struct

    void setData(int data)
```

```
{
    this->data = data; // this->data is the member, data is the local parameter
}
};
```

У этого Something класса есть элемент с именем data. Параметр функции setData() также имеет имя data. Внутри setData() функции data ссылается на параметр функции (поскольку параметр функции затеняет элемент данных), поэтому, если мы хотим сослаться на data элемент, мы используем this->data.

Некоторые разработчики предпочитают явно добавлять this-> ко всем элементам класса, чтобы было понятно, что они ссылаются на элемент. Мы рекомендуем этого не делать, так как это делает код менее читаемым без особой пользы. Использование префикса «m_» — более лаконичный способ отличить закрытые переменные-элементы от переменных, не являющихся элементами (локальных).

Возвращающийся *this

Во-вторых, иногда бывает полезно, чтобы функция-элемент возвращала неявный объект в качестве возвращаемого значения. Основная причина этого заключается в том, что функции-элементы можно «объединять» в цепочку, чтобы несколько функций-элементов можно было вызывать для одного и того же объекта в одном выражении! Это называется объединением функций (или объединением методов).

Рассмотрим распространённый пример, в котором вы выводите несколько фрагментов текста с помощью std::cout:

```
std::cout << "Hello, " << userName;
```

Компилятор оценивает приведенный выше фрагмент следующим образом:

```
(std::cout << "Hello, ") << userName;
```

Во-первых, operator<< использует std::cout и строковый литерал "Hello, " для вывода "Hello, " на консоль. Однако, поскольку это часть выражения, operator<< также должен возвращать значение (или void). Если бы operator<< возвращал void, то в результате получилось бы следующее частично вычисленное выражение:

```
void{} << userName;
```

что явно не имеет смысла (и компилятор выдаст ошибку). Вместо этого operator<< возвращает переданный объект потока, которым в данном случае является std::cout. Таким образом, после вычисления первого operator<< мы получаем:

```
(std::cout) << userName;
```

который затем выводит имя пользователя.

Таким образом, нам нужно указать std::cout только один раз, а затем мы можем объединить столько фрагментов текста с помощью operator<<, сколько захотим. Каждый вызов operator<< возвращает std::cout, поэтому при следующем вызове operator<<std::cout используется в качестве левого операнда.

Мы можем реализовать такое поведение и в наших функциях-элементах. Рассмотрим следующий класс:

```
class Calc
{
private:
    int m_value{};

public:

    void add(int value) { m_value += value; }
    void sub(int value) { m_value -= value; }
    void mult(int value) { m_value *= value; }

    int getValue() const { return m_value; }
};
```

Если бы вы хотели прибавить 5, вычесть 3 и умножить на 4, вам пришлось бы сделать следующее:

```
#include <iostream>

int main()
{
    Calc calc{};
    calc.add(5); // returns void
    calc.sub(3); // returns void
    calc.mult(4); // returns void

    std::cout << calc.getValue() << '\n';

    return 0;
}
```

Однако, если мы сделаем так, чтобы каждая функция возвращала `*this` по ссылке, мы сможем объединять вызовы в цепочку. Вот новая версия `Calc` с «объединяемыми» функциями:

```
class Calc
{
private:
    int m_value{};

public:
    Calc& add(int value) { m_value += value; return *this; }
    Calc& sub(int value) { m_value -= value; return *this; }
    Calc& mult(int value) { m_value *= value; return *this; }

    int getValue() const { return m_value; }
};
```

Обратите внимание, что `add()`, `sub()` и `mult()` теперь возвращаются `*this` по ссылке. Следовательно, это позволяет нам выполнить следующее:

```
#include <iostream>

int main()
{
    Calc calc{};
    calc.add(5).sub(3).mult(4); // method chaining

    std::cout << calc.getValue() << '\n';

    return 0;
}
```

Мы эффективно объединили три строки в одно выражение! Давайте подробнее рассмотрим, как это работает.

Сначала вызывается `calc.add(5)`, которая добавляет 5 к `m_value`. `add()` Затем возвращается ссылка на `*this`, которая является ссылкой на неявный объект `calc`, поэтому `calc` будет объектом, используемым в последующем вычислении. Далее вычисляется `calc.sub(3)`, которая вычитает 3 из `m_value` и снова возвращает `calc`. Наконец, `calc.mult(4)` умножает `m_value` на 4 и возвращает `calc`, который не используется далее и поэтому игнорируется.

Поскольку каждая функция изменяла `calc` по мере выполнения, `m_value` в `calc` теперь содержит значение $((0 + 5) - 3) * 4$, которое равно 8.

Это, вероятно, наиболее распространённое явное использование `this` и то, что вам следует учитывать всякий раз, когда есть смысл использовать функции-элементы с возможностью объединения.

Поскольку `this` всегда указывает на неявный объект, нам не нужно проверять, является ли он нулевым указателем, прежде чем разыменовывать его.

Возврат класса к состоянию по умолчанию

Если в вашем классе есть конструктор по умолчанию, вам может быть интересно узнать, как вернуть существующий объект в исходное состояние.

Как отмечалось в предыдущих уроках (Делегирование конструкторов), конструкторы предназначены только для инициализации новых объектов и не должны вызываться напрямую. В противном случае это приведёт к непредвиденным результатам.

Лучший способ вернуть класс в состояние по умолчанию — создать функцию-элемент `reset()` и с помощью этой функции создать новый объект (используя конструктор по умолчанию), а затем присвоить этот новый объект текущему неявному объекту, например так:

```
void reset()
{
    *this = {}; // value initialize a new object and overwrite the implicit object
}
```

Вот полная программа, демонстрирующая эту `reset()` функцию в действии:

```
#include <iostream>

class Calc
{
private:
    int m_value{};

public:
    Calc& add(int value) { m_value += value; return *this; }
    Calc& sub(int value) { m_value -= value; return *this; }
    Calc& mult(int value) { m_value *= value; return *this; }

    int getValue() const { return m_value; }

    void reset() { *this = {}; }
};

int main()
{
    Calc calc{};
    calc.add(5).sub(3).mult(4);

    std::cout << calc.getValue() << '\n'; // prints 8

    calc.reset();

    std::cout << calc.getValue() << '\n'; // prints 0

    return 0;
}
```

this и объекты const

Для функций-элементов, не являющихся константными, `this` — это константный указатель на неконстантное значение (то есть `this` не может указывать на что-то другое, но объект, на который он указывает, может быть изменён). Для константных функций-элементов `this` — это константный указатель на константное значение (то есть указатель не может указывать на что-то другое, а объект, на который он указывает, не может быть изменён).

Ошибки, возникающие при попытке вызвать функцию-элемент, не являющуюся константой, для объекта-константы, могут быть немного загадочными:

```
ошибка C2662: 'int Something::getValue(void)': невозможно преобразовать указатель  
'this' из 'const Something' в 'Something &'  
ошибка: передача 'const Something' в качестве аргумента 'this' игнорирует квалификаторы  
[-fpermissive]
```

Когда мы вызываем функцию-элемент, не являющуюся константной, для константного объекта, неявным параметром `this` функции является константный указатель на неконстантный объект. Но аргумент имеет тип константного указателя на константный объект. Преобразование указателя на константный объект в указатель на неконстантный объект требует удаления квалификатора `const`, что не может быть сделано неявно. Ошибка компиляции, возникающая в некоторых компиляторах, отражает недовольство компилятора тем, что его просят выполнить такое преобразование.

Почему `this` указатель, а не ссылка

Поскольку указатель `this` всегда указывает на неявный объект (и никогда не может быть нулевым указателем, если только мы не сделали что-то, что привело к неопределённому поведению), вам может быть интересно, почему `this` является указателем, а не ссылкой. Ответ прост: когда `this` был добавлен в C++, ссылок ещё не существовало.

Если бы `this` был добавлен в язык C++ сегодня, он, несомненно, был бы ссылкой, а не указателем. В других более современных языках, похожих на C++, таких как Java и C#, `this` реализован как ссылка.

Классы и заголовочные файлы

Все классы, которые мы написали до сих пор, были достаточно простыми, чтобы мы могли реализовать функции-элементы непосредственно в самом определении класса. Например, вот простой Date класс, в котором все функции-элементы определены внутри Date определения класса:

```
#include <iostream>

class Date
{
private:
    int m_year{};
    int m_month{};
    int m_day{};

public:
    Date(int year, int month, int day)
        : m_year { year }
        , m_month { month }
        , m_day { day }
    {
    }

    void print() const { std::cout << "Date(" << m_year << ", " << m_month << ", " <<
m_day << ")\n"; }

    int getYear() const { return m_year; }
    int getMonth() const { return m_month; }
    int getDay() const { return m_day; }
};

int main()
{
    Date d { 2015, 10, 14 };
    d.print();

    return 0;
}
```

Однако по мере того, как классы становятся длиннее и сложнее, наличие всех определений функций-элементов внутри класса может усложнить управление классом и работу с ним. Использование уже написанного класса требует лишь понимания его публичного интерфейса (публичных функций-элементов), а не того, как класс работает «под капотом». Реализация функций-элементов загромождает публичный интерфейс деталями, которые не имеют отношения к использованию класса.

Чтобы решить эту проблему, C++ позволяет отделить «объявление» класса от «реализации» путём определения функций-элементов вне определения класса.

Вот тот же класс Date из примера выше, но с конструктором и функциями-элементами print() за пределами определения класса. Обратите внимание, что прототипы этих функций-элементов по-прежнему находятся внутри определения класса (поскольку эти функции должны быть объявлены как часть определения типа класса), но фактическая реализация вынесена за пределы:

```
#include <iostream>

class Date
{
private:
    int m_year{};
    int m_month{};
    int m_day{};
```

```

public:
    Date(int year, int month, int day); // constructor declaration

    void print() const; // print function declaration

    int getYear() const { return m_year; }
    int getMonth() const { return m_month; }
    int getDay() const { return m_day; }
};

Date::Date(int year, int month, int day) // constructor definition
    : m_year{ year }
    , m_month{ month }
    , m_day{ day }
{
}

void Date::print() const // print function definition
{
    std::cout << "Date(" << m_year << ", " << m_month << ", " << m_day << ")\n";
};

int main()
{
    const Date d{ 2015, 10, 14 };
    d.print();

    return 0;
}

```

Функции-элементы могут быть определены вне определения класса так же, как и функции, не являющиеся элементами класса. Единственное отличие состоит в том, что перед именами функций-элементов мы должны указывать имя типа класса (в данном случае Date::) в качестве префикса, чтобы компилятор знал, что мы определяем функцию-элемент этого типа класса, а не функцию, не являющуюся элементом класса.

Обратите внимание, что мы оставили функции доступа, определённые внутри определения класса. Поскольку функции доступа обычно состоят из одной строки, определение этих функций внутри определения класса добавляет минимум беспорядка, в то время как их перемещение за пределы определения класса привело бы к появлению множества дополнительных строк кода. По этой причине определения функций доступа (и других тривиальных однострочных функций) часто оставляют внутри определения класса.

Помещение определений классов в заголовочный файл

Если вы определяете класс внутри исходного файла (.cpp), этот класс можно использовать только в этом конкретном исходном файле. В более крупных программах мы часто хотим использовать классы, которые пишем в нескольких исходных файлах.

Объявления функций можно размещать в заголовочных файлах. Затем вы можете включать эти объявления функций в несколько файлов кода (или даже в несколько проектов) через директиву #include. Классы ничем не отличаются от этого. Определения классов можно размещать в заголовочных файлах, а затем включать в любые другие файлы, в которых нужно использовать тип класса.

В отличие от функций, для использования которых достаточно предварительного объявления, компилятору обычно требуется полное определение класса (или любого другого типа, определяемого программой), чтобы использовать этот тип. Это связано с тем, что компилятору нужно понимать, как объявлены элементы, чтобы обеспечить их правильное использование, а также уметь вычислять размер объектов этого типа, чтобы создавать их экземпляры. Поэтому наши заголовочные файлы обычно содержат полное определение класса, а не только предварительное объявление.

Присвоение имен заголовочным файлам вашего класса и файлам кода

Чаще всего классы определяются в заголовочных файлах с тем же именем, что и у класса, а любые функции-элементы, определённые вне класса, помещаются в файл .cpp с тем же именем, что и у класса.

Вот наш класс Date, разделённый на файлы .cpp и .h:

Date.h

```
#ifndef DATE_H
#define DATE_H

class Date
{
private:
    int m_year{};
    int m_month{};
    int m_day{};

public:
    Date(int year, int month, int day);

    void print() const;

    int getYear() const { return m_year; }
    int getMonth() const { return m_month; }
    int getDay() const { return m_day; }
};

#endif
```

Date.cpp:

```
#include <iostream>
#include "Date.h"

Date::Date(int year, int month, int day) // constructor definition
    : m_year{ year }
    , m_month{ month }
    , m_day{ day }
{
}

void Date::print() const // print function definition
{
    std::cout << "Date(" << m_year << ", " << m_month << ", " << m_day << ")\n";
};
```

Теперь любой другой заголовочный файл или файл с кодом, который хочет использовать класс Date, может просто #include "Date.h". Обратите внимание, что Date.cpp также необходимо скомпилировать в любом проекте, использующем Date.h, чтобы компоновщик мог связать вызовы функций-элементов с их определениями.

Наилучшая практика

Лучше всего размещать определения классов в заголовочном файле с тем же именем, что и у класса. Простые функции-элементы (например, функции доступа, конструкторы с пустыми телами и т. д.) можно определить внутри определения класса.

Рекомендуется определять нетривиальные функции-элементы в исходном файле с тем же именем, что и у класса.

Не нарушает ли определение класса в заголовочном файле правило единого определения, если заголовочный файл включается более одного раза?

Типы не подпадают под действие части правила единого определения (ODR), которая гласит, что в программе может быть только одно определение. Поэтому нет проблем с #включением определений классов в несколько единиц трансляции. Если бы они были, классы не были бы так полезны.

Включение определения класса более одного раза в одну единицу трансляции по-прежнему является нарушением ODR. Однако защита заголовков (или `#pragma once`) предотвратит это.

Встроенные функции (inline)

Inline-функция — это специальная функция, тело которой подставляется непосредственно в каждую точку вызова вместо обычного вызова функции. Это похоже на работу параметризованных макросов в C, но с более безопасным и контролируемым поведением.

Зачем нужны inline-функции?

Основная причина использования inline-функций — повышение эффективности программы. При обычном вызове функции происходит:

- Сохранение текущего состояния
- Передача аргументов в стек
- Переход к коду функции
- Выполнение кода
- Возврат результата
- Восстановление состояния

Все эти операции требуют дополнительных тактов процессора. При использовании inline-функции весь этот процесс пропускается, и код функции просто встраивается в место вызова, что ускоряет выполнение программы.

Как создать inline-функцию?

Существует один основной способ создания inline-функции:

```
inline тип_возврата имя_функции(параметры) {  
    // тело функции  
}
```

Например:

```
inline int add(int a, int b) {  
    return a + b;  
}
```

Важные особенности:

Inline — это рекомендация компилятору, а не требование. Компилятор может проигнорировать эту подсказку.

Не все функции могут быть inline:

- Функции с циклами
- Функции с оператором `goto`
- Функции с исключениями
- Функции, работающие с объектами, имеющими деструкторы

Inline-функции обычно небольшие. Для больших функций накладные расходы на встраивание могут превысить выгоду от оптимизации.

Пример использования:

```
#include <iostream>

class Counter {
private:
    int value;
public:
    inline int getValue() { return value; }
    inline void setValue(int val) { value = val; }
};

int main() {
    Counter c;
    c.setValue(42);
    std::cout << c.getValue() << std::endl;
    return 0;
}
```

В этом примере функции `getValue()` и `setValue()` будут встроены в места их вызова, что ускорит выполнение программы по сравнению с обычными функциями.

Когда использовать inline-функции:

- Для маленьких функций-аксессоров (геттеров и сеттеров)
- Для математических функций с простыми вычислениями
- Для функций, которые вызываются очень часто

Когда не использовать:

- Для больших функций
- Для функций с рекурсией
- Для функций с сложными операциями

Inline-функции — это мощный инструмент оптимизации, но его нужно использовать разумно, учитывая специфику вашей программы и требования к производительности.

Встроенные функции-элементы !!!!!

Функции-элементы не освобождаются от ODR, поэтому вам может быть интересно, как избежать нарушений ODR, когда функции-элементы определены в заголовочном файле (который может быть включён более чем в одну единицу трансляции).

Функции-элементы, определённые внутри определения класса, неявно встраиваются. Встроенные функции не подпадают под правило одного определения в программе.

Функции-элементы, определённые вне определения класса, не встраиваются неявно (и, следовательно, подчиняются правилу одного определения для каждой программы). Именно поэтому такие функции обычно определяются в файле кода (где они будут иметь только одно определение для всей программы).

В качестве альтернативы функции-элементы, определённые вне определения класса, могут быть оставлены в заголовочном файле, если они объявлены встроенными (с использованием ключевого слова `inline`). Вот наш заголовочный файл `Date.h` с функциями-элементами, определёнными вне класса, отмеченными `inline`:

`Date.h`:

```
#ifndef DATE_H
#define DATE_H

#include <iostream>

class Date
```

```

{
private:
    int m_year{};
    int m_month{};
    int m_day{};

public:
    Date(int year, int month, int day);

    void print() const;

    int getYear() const { return m_year; }
    int getMonth() const { return m_month; }
    int getDay() const { return m_day; }
};

inline Date::Date(int year, int month, int day) // now inline
    : m_year{ year }
    , m_month{ month }
    , m_day{ day }
{
}

inline void Date::print() const // now inline
{
    std::cout << "Date(" << m_year << ", " << m_month << ", " << m_day << ")\n";
};

#endif

```

Этот Date.h можно без проблем включить в несколько блоков кода.

Ключевое понимание

Функции, определённые внутри определения класса, неявно встраиваются, что позволяет включать их в несколько файлов кода без нарушения ODR.

Функции, определённые вне определения класса, не встраиваются автоматически. Их можно сделать встроенными с помощью ключевого слова inline

Встроенное расширение функций-элементов

Для выполнения встроенного расширения компилятор должен видеть полное определение функции. Чаще всего такие функции (например, функции доступа) определяются внутри определения класса. Однако, если вы хотите определить функцию-элемент вне определения класса, но при этом хотите, чтобы она подходила для встроенного расширения, вы можете определить её как встроенную функцию сразу под определением класса (в том же заголовочном файле). Таким образом, определение функции будет доступно всем, кто #включает заголовочный файл.

Так почему бы не поместить все в заголовочный файл?

У вас может возникнуть соблазн поместить все определения функций-элементов в заголовочный файл либо внутри определения класса, либо в виде встроенных функций под определением класса. Хотя это позволит выполнить компиляцию, у такого подхода есть несколько недостатков.

Во-первых, как упоминалось выше, определение элементов внутри определения класса усложняет его структуру.

Во-вторых, если вы измените какой-либо код в заголовке, вам нужно будет перекомпилировать все файлы, которые включают этот заголовок. Это может привести к цепной реакции, когда одно незначительное изменение потребует перекомпиляции всей программы. Стоимость перекомпиляции может значительно варьироваться: сборка небольшого проекта может занять всего минуту или меньше, в то время как сборка крупного коммерческого проекта может занять несколько часов.

И наоборот, если вы измените код в файле .cpp, перекомпилировать нужно будет только этот файл .cpp. Поэтому, если есть выбор, лучше помещать нетривиальный код в файл .cpp.

В некоторых случаях имеет смысл нарушить общепринятую практику и поместить определение класса в заголовочный файл, а нетривиальные функции-элементы — в файл с кодом.

Во-первых, для небольшого класса, который используется только в одном файле кода и не предназначен для многократного использования, вы можете предпочесть определить класс (и все функции-элементы) непосредственно в том единственном файле .cpp, в котором он используется. Это поможет понять, что класс используется только в этом файле и не предназначен для более широкого применения. Вы всегда можете переместить класс в отдельный файл заголовка/кода, если позже решите использовать его более чем в одном файле или обнаружите, что определения класса и функций-элементов загромождают исходный файл.

Во-вторых, если у класса есть лишь небольшое количество нетривиальных функций-элементов, которые вряд ли изменятся, создание файла .cpp, содержащего всего одно или два определения, может оказаться нецелесообразным (поскольку это загромождаст ваш проект). В таких случаях может быть предпочтительнее создать функции-элементы inline и разместить их под определением класса в заголовочном файле.

В-третьих, в современном C++ классы или библиотеки всё чаще распространяются как «только заголовочные файлы», то есть весь код класса или библиотеки размещается в заголовочном файле. Это делается в первую очередь для упрощения распространения и использования таких файлов, поскольку заголовочный файл нужно только #include, а файл с кодом нужно явно добавлять в каждый проект, в котором он используется, чтобы его можно было скомпилировать. Если вы намеренно создаёте класс или библиотеку только с заголовочными файлами для распространения, все нетривиальные функции-элементы можно сделать inline и разместить в заголовочном файле под определением класса.

Наконец, для шаблонных классов функции-элементы шаблона, определённые вне класса, почти всегда определяются внутри заголовочного файла под определением класса. Как и в случае с функциями-элементами шаблона, компилятору необходимо видеть полное определение шаблона, чтобы создать его экземпляр.

В следующих уроках большинство наших классов будут определяться в одном файле .cpp, а все функции будут реализованы непосредственно в определении класса. Это сделано для того, чтобы примеры были лаконичными и их было легко скомпилировать самостоятельно. В реальных проектах классы гораздо чаще помещаются в собственные файлы кода и заголовочных файлов, и вам следует привыкнуть к этому.

Аргументы по умолчанию для функций-элементов

Аргументы по умолчанию мы обсудили наилучшую практику для аргументов по умолчанию в функциях, не являющихся элементами класса: «Если у функции есть предварительное объявление (особенно в заголовочном файле), поместите аргумент по умолчанию туда. В противном случае поместите аргумент по умолчанию в определение функции».

Поскольку функции-элементы всегда объявляются (или определяются) как часть определения класса, лучший способ работы с функциями-элементами на самом деле проще: всегда помещайте аргумент по умолчанию внутрь определения класса.

Наилучшая практика

Поместите любые аргументы по умолчанию для функций-элементов внутри определения класса.

Библиотеки

В своих программах вы использовали классы, входящие в стандартную библиотеку, например std::string. Чтобы использовать эти классы, вам нужно просто #include соответствующий заголовочный файл (например, #include <string>). Обратите внимание, что вам не нужно добавлять в свои проекты какие-либо файлы кода (например, string.cpp или iostream.cpp).

Заголовочные файлы содержат объявления, необходимые компилятору для проверки синтаксической правильности программ, которые вы пишете. Однако реализации классов, входящих в стандартную библиотеку C++, содержатся в предварительно скомпилированном файле, который автоматически подключается на этапе компоновки. Вы никогда не увидите этот код.

Многие программные пакеты с открытым исходным кодом предоставляют файлы .h и .cpp для компиляции в вашу программу. Однако большинство коммерческих библиотек предоставляют только файлы .h и предварительно скомпилированный файл библиотеки. Для этого есть несколько причин:

- 1) быстрее связать предварительно скомпилированную библиотеку, чем перекомпилировать её каждый раз, когда она вам нужна;
- 2) одна копия предварительно скомпилированной библиотеки может использоваться многими приложениями, в то время как скомпилированный код компилируется в каждый исполняемый файл, который его использует (что увеличивает размер файлов);
- 3) соображения, связанные с интеллектуальной собственностью (вы же не хотите, чтобы люди воровали ваш код).

Хотя вы, скорее всего, какое-то время не будете создавать и распространять собственные библиотеки, разделение классов на заголовочные и исходные файлы — это не только хорошая практика, но и упрощающее создание собственных библиотек. Создание собственных библиотек выходит за рамки этих руководств, но разделение объявления и реализации является обязательным условием для этого, если вы хотите распространять предварительно скомпилированные двоичные файлы.

Вложенные типы (типы-элементы)

Рассмотрим следующую короткую программу:

```
#include <iostream>

enum class FruitType
{
    apple,
    banana,
    cherry
};

class Fruit
{
private:
    FruitType m_type { };
    int m_percentageEaten { 0 };

public:
    Fruit(FruitType type) :
        m_type { type }
    {
    }

    FruitType getType() { return m_type; }
    int getPercentageEaten() { return m_percentageEaten; }

    bool isCherry() { return m_type == FruitType::cherry; }
};

int main()
{
    Fruit apple { FruitType::apple };

    if (apple.getType() == FruitType::apple)
        std::cout << "I am an apple";
    else
        std::cout << "I am not an apple";

    return 0;
}
```

В этой программе нет ничего плохого. Но поскольку `enum class FruitType` предназначен для использования в сочетании с классом `Fruit`, его существование независимо от класса заставляет нас догадываться, как они связаны.

Вложенные типы (типы-элементы)

До сих пор мы рассматривали типы классов с двумя разными видами элементов: элементами данных и функциями-элементами. В нашем примере `Fruit` класс содержит и то, и другое.

Типы классов поддерживают другой вид элементов: вложенные типы (также называемые типами элементов). Чтобы создать вложенный тип, вы просто определяете тип внутри класса с помощью соответствующего спецификатора доступа.

Вот та же программа, что и выше, но переписанная с использованием вложенного типа, определённого внутри класса `Fruit`:

```
#include <iostream>
```

```

class Fruit
{
public:
    // FruitType has been moved inside the class, under the public access specifier
    // We've also renamed it Type and made it an enum rather than an enum class
    enum Type
    {
        apple,
        banana,
        cherry
    };

private:
    Type m_type {};
    int m_percentageEaten { 0 };

public:
    Fruit(Type type) :
        m_type { type }
    {
    }

    Type getType() { return m_type; }
    int getPercentageEaten() { return m_percentageEaten; }

    bool isCherry() { return m_type == cherry; } // Inside members of Fruit, we no
longer need to prefix enumerators with FruitType::
};

int main()
{
    // Note: Outside the class, we access the enumerators via the Fruit:: prefix now
    Fruit apple { Fruit::apple };

    if (apple.getType() == Fruit::apple)
        std::cout << "I am an apple";
    else
        std::cout << "I am not an apple";

    return 0;
}

```

Здесь есть несколько вещей, на которые стоит обратить внимание.

Во-первых, обратите внимание, что FruitType теперь определяется внутри класса, где он был переименован в Type по причинам, которые мы вскоре обсудим.

Во-вторых, вложенный тип Type был определён в верхней части класса. Имена вложенных типов должны быть полностью определены, прежде чем их можно будет использовать, поэтому обычно они определяются в первую очередь.

Наилучшая практика

Определите все вложенные типы в верхней части вашего типа класса.

В-третьих, вложенные типы подчиняются обычным правилам доступа. Type определяется с помощью спецификатора доступа public, так что к имени типа и перечисляемым значениям можно получить прямой доступ.

В-четвёртых, типы классов действуют как область видимости для объявленных внутри них имён, подобно пространствам имён. Поэтому полное имя Type — Fruit::Type, а полное имя apple-перечисления — Fruit::apple.

Внутри элементов класса нам не нужно использовать полное имя. Например, в функции-элементе `isCherry()` мы обращаемся к `cherry` перечислителю без `Fruit::` квалификатора области видимости.

Вне класса мы должны использовать полное имя (например, `Fruit::apple`). Мы переименовали `FruitType` в `Type`, чтобы обращаться к нему как `Fruit::Type` (а не как к более избыточному `Fruit::FruitType`).

Наконец, мы изменили наш перечисляемый тип с ограниченного на неограниченный. Поскольку сам класс теперь выступает в качестве области видимости, использование ограниченного перечислителя также является избыточным. Переход на неограниченный перечислитель означает, что мы можем обращаться к перечислителям как `Fruit::apple` вместо более длинного `Fruit::Type::apple` варианта, который пришлось бы использовать, если бы перечислитель был ограничен.

Вложенные определения типов и псевдонимы типов

Типы классов также могут содержать вложенные определения типов или псевдонимы типов:

```
#include <iostream>
#include <string>
#include <string_view>

class Employee
{
public:
    using IDType = int;

private:
    std::string m_name{};
    IDType m_id{};
    double m_wage{};

public:
    Employee(std::string_view name, IDType id, double wage)
        : m_name { name }
        , m_id { id }
        , m_wage { wage }
    {
    }

    const std::string& getName() { return m_name; }
    IDType getId() { return m_id; } // can use unqualified name within class
};

int main()
{
    Employee john { "John", 1, 45000 };
    Employee::IDType id { john.getId() }; // must use fully qualified name outside
class

    std::cout << john.getName() << " has id: " << id << '\n';

    return 0;
}
```

Это печатает:

```
John has id: 1
```

Обратите внимание, что внутри класса мы можем использовать просто `IDType`, но вне класса мы должны использовать полное имя `Employee::IDType`.

Вложенные классы и доступ к элементам внешнего класса

Довольно редко классы содержат другие классы в качестве вложенных типов, но это возможно. В C++ вложенный класс не имеет доступа к `this` указателю внешнего (содержащего) класса, поэтому вложенные классы не могут напрямую обращаться к элементам внешнего класса. Это связано с тем, что вложенный класс может быть создан независимо от внешнего класса (и в таком случае не будет элементов внешнего класса, к которым можно было бы обратиться!)

Однако, поскольку вложенные классы являются элементами внешнего класса, они могут получить доступ к любым закрытым элементам внешнего класса, которые находятся в области видимости.

Давайте проиллюстрируем это на примере:

```
#include <iostream>
#include <string>
#include <string_view>

class Employee
{
public:
    using IDType = int;

    class Printer
    {
public:
        void print(const Employee& e) const
        {
            // Printer can't access Employee's `this` pointer
            // so we can't print m_name and m_id directly
            // Instead, we have to pass in an Employee object to use
            // Because Printer is a member of Employee,
            // we can access private members e.m_name and e.m_id directly
            std::cout << e.m_name << " has id: " << e.m_id << '\n';
        }
    };

private:
    std::string m_name{};
    IDType m_id{};
    double m_wage{};

public:
    Employee(std::string_view name, IDType id, double wage)
        : m_name{ name }
        , m_id{ id }
        , m_wage{ wage }
    {
    }

    // removed the access functions in this example (since they aren't used)
};

int main()
{
    const Employee john{ "John", 1, 45000 };
    const Employee::Printer p{}; // instantiate an object of the inner class
    p.print(john);

    return 0;
}
```

Это печатает:

```
John has id: 1
```

Это один из случаев, когда вложенные классы используются чаще. В стандартной библиотеке большинство классов итераторов реализованы как вложенные классы контейнера, для которого они предназначены. Например, `std::string::iterator` реализован как вложенный класс `std::string`. Мы рассмотрим итераторы в следующей главе.

Вложенные типы и прямые объявления

Вложенный тип может быть предварительно объявлен внутри класса, в который он вложен. Вложенный тип может быть определён позже, либо внутри вложенного класса, либо вне его. Например:

```
#include <iostream>

class outer
{
public:
    class inner1;    // okay: forward declaration inside the enclosing class okay
    class inner1{};  // okay: definition of forward declared type inside the enclosing class
    class inner2;    // okay: forward declaration inside the enclosing class okay
};

class inner2 // okay: definition of forward declared type outside the enclosing class
{
};

int main()
{
    return 0;
}
```

Однако вложенный тип не может быть предварительно объявлен до определения включающего его класса.

```
#include <iostream>

class outer;          // okay: can forward declare non-nested type
class outer::inner1;  // error: can't forward declare nested type prior to outer class definition

class outer
{
public:
    class inner1{};    // note: nested type declared here
};

class outer::inner1;  // okay (but redundant) since nested type has already been
                    // declared as part of outer class definition

int main()
{
    return 0;
}
```

Хотя вы можете объявить вложенный тип после определения включающего класса, поскольку включающий класс уже содержит объявление вложенного типа, делать это необязательно.

Введение в деструкторы

Проблема с оч0438сткой

Допустим, вы пишете программу, которой нужно отправлять какие-то данные по сети. Однако установление соединения с сервером обходится дорого, поэтому вы хотите собрать несколько наборов данных, а затем отправить их все сразу. Такой класс может быть структурирован следующим образом:

```
// This example won't compile because it is (intentionally) incomplete
class NetworkData
{
private:
    std::string m_serverName{};
    DataStore m_dataQueue{};

public:
    NetworkData(std::string_view serverName)
        : m_serverName { serverName }
    {
    }

    void addData(std::string_view data)
    {
        m_dataQueue.add(data);
    }

    void sendData()
    {
        // connect to server
        // send all data
        // clear data
    }
};

int main()
{
    NetworkData n("someipAddress");

    n.addData("somedata1");
    n.addData("somedata2");

    n.sendData();

    return 0;
}
```

Однако у этого NetworkData есть потенциальная проблема. Она заключается в том, что sendData() должен быть явно вызван до завершения работы программы. Если пользователь NetworkData забудет это сделать, данные не будут отправлены на сервер и будут потеряны при выходе из программы. Вы можете сказать: «Ну, это несложно запомнить!», и в этом конкретном случае вы будете правы. Но рассмотрим чуть более сложный пример, например, эту функцию:

```
bool someFunction()
{
    NetworkData n("someipAddress");

    n.addData("somedata1");
    n.addData("somedata2");

    if (someCondition)
        return false;
}
```

```
n.sendData();  
return true;  
}
```

В этом случае, если `someCondition` равно `true`, то функция завершится раньше, и `sendData()` не будет вызвана. Эту ошибку совершить проще, потому что вызов `sendData()` присутствует, просто программа не переходит к нему во всех случаях.

Если обобщить эту проблему, то классы, использующие ресурсы (чаще всего память, но иногда файлы, базы данных, сетевые подключения и т. д.), часто необходимо явно отправлять или закрывать до того, как объект класса, использующий их, будет уничтожен. В других случаях мы можем захотеть выполнить какие-то действия перед уничтожением объекта, например, записать информацию в файл журнала или отправить данные на сервер. Термин «очистка» часто используется для обозначения любого набора задач, которые класс должен выполнить перед уничтожением объекта класса, чтобы он работал должным образом. Если мы полагаемся на то, что пользователь такого класса вызовет функцию очистки до уничтожения объекта, мы, скорее всего, столкнёмся с ошибками.

Но зачем вообще требовать от пользователя подтверждения этого? Если объект уничтожается, то мы знаем, что в этот момент необходимо выполнить очистку. Должна ли эта очистка выполняться автоматически?

Деструкторы спешат на помощь

На уроке Знакомство с конструкторами мы рассмотрели конструкторы — специальные функции-элементы, которые вызываются при создании объекта неагрегированного типа класса. Конструкторы используются для инициализации переменных-элементов и выполнения любых других задач по настройке, необходимых для подготовки объектов класса к использованию.

Аналогичным образом, у классов есть другой тип специальной функции-элемента, которая вызывается автоматически при уничтожении объекта неагрегированного типа класса. Эта функция называется деструктором. Деструкторы предназначены для того, чтобы класс мог выполнить необходимую очистку перед уничтожением объекта класса.

Присвоение имени деструктору

Как и конструкторы, деструкторы имеют определенные правила именования:

- Деструктор должен иметь то же имя, что и класс, с префиксом тильды (~).
- Деструктор не может принимать аргументы.
- Деструктор не имеет возвращаемого типа.
- Класс может иметь только один деструктор.

Как правило, не следует вызывать деструктор явно (поскольку он будет вызван автоматически при уничтожении объекта), поскольку редко бывают случаи, когда нужно очистить объект более одного раза.

Деструкторы могут безопасно вызывать другие функции-элементы, поскольку объект не уничтожается до выполнения деструктора.

Пример деструктора

```
#include <iostream>  
  
class Simple  
{  
private:  
    int m_id {};  
  
public:  
    Simple(int id)  
        : m_id { id }  
    {  
        std::cout << "Constructing Simple " << m_id << '\n';  
    }  
};
```

```

    }

    ~Simple() // here's our destructor
    {
        std::cout << "Destructing Simple " << m_id << '\n';
    }

    int getID() const { return m_id; }
};

int main()
{
    // Allocate a Simple
    Simple simple1{ 1 };
    {
        Simple simple2{ 2 };
    } // simple2 dies here

    return 0;
} // simple1 dies here

```

Эта программа выдает следующий результат:

```

Constructing Simple 1
Constructing Simple 2
Destructing Simple 2
Destructing Simple 1

```

Обратите внимание, что при уничтожении каждого объекта Simple вызывается деструктор, который выводит сообщение. «Уничтожение Simple 1» выводится после «Уничтожения Simple 2», потому что simple2 был уничтожен до конца функции, а simple1 не был уничтожен до конца main().

Помните, что статические переменные (включая глобальные и статические локальные переменные) создаются при запуске программы и уничтожаются при её завершении.

Улучшение программы NetworkData

Вернёмся к нашему примеру в начале урока. Мы можем избавить пользователя от необходимости явно вызывать sendData() с помощью деструктора, который вызывает эту функцию:

```

class NetworkData
{
private:
    std::string m_serverName{};
    DataStore m_dataQueue{};

public:
    NetworkData(std::string_view serverName)
        : m_serverName { serverName }
    {
    }

    ~NetworkData()
    {
        sendData(); // make sure all data is sent before object is destroyed
    }

    void addData(std::string_view data)
    {
        m_dataQueue.add(data);
    }
}

```

```

    void sendData()
    {
        // connect to server
        // send all data
        // clear data
    }
};

int main()
{
    NetworkData n("someipAddress");

    n.addData("somedata1");
    n.addData("somedata2");

    return 0;
}

```

С таким деструктором наш объект NetworkData всегда будет отправлять все имеющиеся у него данные до того, как объект будет уничтожен! Очистка происходит автоматически, а значит, меньше вероятность ошибок и меньше поводов для размышлений.

Неявный деструктор

Если у объекта неагрегированного класса нет объявленного пользователем деструктора, компилятор сгенерирует деструктор с пустым телом. Такой деструктор называется неявным деструктором и фактически является просто заполнителем.

Если вашему классу не нужно выполнять какую-либо очистку при уничтожении, можно вообще не определять деструктор и позволить компилятору сгенерировать неявный деструктор для вашего класса.

Предупреждение о функции std::exit()

Функция std::exit() может использоваться для немедленного завершения программы. При немедленном завершении программы она просто завершается. Локальные переменные не уничтожаются в первую очередь, и поэтому деструкторы не вызываются. Будьте осторожны, если вы полагаетесь на деструкторы для выполнения необходимой очистки в таком случае.

Для продвинутых читателей

Необработанные исключения также приведут к завершению работы программы и могут не привести к очистке стека перед этим. Если очистка стека не произойдёт, деструкторы не будут вызваны до завершения работы программы.

Статические переменные- элементы

Статические переменные создаются в начале программы и уничтожаются в конце программы. Такие переменные сохраняют свои значения, даже если выходят за пределы области видимости.

Например:

```
#include <iostream>

int generateID()
{
    static int s_id{ 0 }; // static local variable
    return ++s_id;
}

int main()
{
    std::cout << generateID() << '\n';
    std::cout << generateID() << '\n';
    std::cout << generateID() << '\n';

    return 0;
}
```

Эта программа выводит:

```
1
2
3
```

Обратите внимание, что статическая локальная переменная `s_id` сохраняла свое значение при нескольких вызовах функции.

Типы классов позволяют использовать ключевое слово `static` ещё в двух случаях: для статических переменных-элементов и статических функций-элементов. К счастью, эти случаи довольно просты. В этом уроке мы поговорим о статических переменных-элементах, а в следующем — о статических функциях-элементах.

Статические переменные-элементы

Прежде чем мы перейдём к ключевому слову `static` применительно к переменным-элементам, рассмотрим следующую структуру:

```
#include <iostream>

struct Something
{
    int value{ 1 };
};

int main()
{
    Something first{};
    Something second{};

    first.value = 2;

    std::cout << first.value << '\n';
    std::cout << second.value << '\n';

    return 0;
}
```

Когда мы создаём экземпляр объекта класса, каждый объект получает собственную копию всех обычных переменных-элементов. В данном случае, поскольку мы объявили два объекта класса `Something`, у нас получается две копии `value`: `first.value` и `second.value`. `first.value` отличается от `second.value`. Следовательно, приведённая выше программа выводит:

```
2
1
```

Переменные-элементы класса можно сделать статическими с помощью ключевого слова `static`. В отличие от обычных переменных-элементов, статические переменные-элементы являются общими для всех объектов класса. Рассмотрим следующую программу, аналогичную предыдущей:

```
#include <iostream>

struct Something
{
    static int s_value; // declare s_value as static (initializer moved below)
};

int Something::s_value{ 1 }; // define and initialize s_value to 1 (we'll discuss this
section below)

int main()
{
    Something first{};
    Something second{};

    first.s_value = 2;

    std::cout << first.s_value << '\n';
    std::cout << second.s_value << '\n';
    return 0;
}
```

Эта программа выдает следующий результат:

```
2
2
```

Поскольку `s_value` является статической переменной-элементом, `s_value` используется всеми объектами класса. Следовательно, `first.s_value` — это та же переменная, что и `second.s_value`. Приведённая выше программа показывает, что значение, которое мы задали с помощью `first`, можно получить с помощью `second`!

Статические элементы не связаны с объектами класса

Хотя вы можете получить доступ к статическим элементам через объекты класса (как показано с помощью `first.s_value` и `second.s_value` в приведённом выше примере), статические элементы существуют, даже если не было создано ни одного объекта класса! Это логично: они создаются в начале программы и уничтожаются в конце, поэтому их время жизни не привязано к объекту класса, как у обычных элементов.

По сути, статические элементы — это глобальные переменные, которые находятся в области видимости класса. Разница между статическим элементом класса и обычной переменной в пространстве имён очень незначительна.

Ключевое понимание

Статические элементы - это глобальные переменные, которые находятся внутри области видимости класса.

Поскольку статический элемент `s_value` существует независимо от каких-либо объектов класса, к нему можно получить прямой доступ, используя имя класса и оператор разрешения области видимости (в данном случае `Something::s_value`):

```

class Something
{
public:
    static int s_value; // declare s_value as static
};

int Something::s_value{ 1 }; // define and initialize s_value to 1 (we'll discuss this
section below)

int main()
{
    // note: we're not instantiating any objects of type Something

    Something::s_value = 2;
    std::cout << Something::s_value << '\n';
    return 0;
}

```

В приведённом выше фрагменте кода `s_value` ссылается на имя класса `Something` напрямую, а не через объект. Обратите внимание, что мы даже не создали экземпляр объекта типа `Something`, но всё равно можем получить доступ к `Something::s_value` и использовать его. Это предпочтительный способ доступа к статическим элементам.

Наилучшая практика

Получите доступ к статическим элементам, используя имя класса и оператор разрешения области видимости (`::`).

Определение и инициализация статических переменных-элементов

Когда мы объявляем статическую переменную-элемент внутри типа класса, мы сообщаем компилятору о существовании статической переменной-элемента, но не определяем её (как при предварительном объявлении). Поскольку статические переменные-элементы по сути являются глобальными переменными, вы должны явно определить (и при необходимости инициализировать) статический элемент вне класса, в глобальной области видимости.

В приведенном выше примере мы делаем это с помощью этой строки:

```
int Something::s_value{ 1 }; // define and initialize s_value to 1
```

Эта строка служит двум целям: она создаёт экземпляр статической переменной-элемента (как и глобальную переменную) и инициализирует её. В данном случае мы задаём значение инициализации 1. Если инициализатор не задан, статические переменные-элементы по умолчанию инициализируются нулём.

Обратите внимание, что это определение статического элемента не контролируется доступом: вы можете определить и инициализировать значение, даже если оно объявлено как закрытое (или защищённое) в классе (поскольку определения не считаются формой доступа).

Для классов, не являющихся шаблонами, если класс определён в заголовочном файле (`.h`), определение статического элемента обычно помещается в связанный с классом файл кода (например, `Something.cpp`). В качестве альтернативы элемент может быть определён как `inline` и размещён под определением класса в заголовочном файле (это полезно для библиотек, состоящих только из заголовочных файлов). Если класс определён в исходном файле (`.cpp`), определение статического элемента обычно размещается непосредственно под классом. Не помещайте определение статического элемента в заголовочный файл (как и в случае с глобальной переменной, если этот заголовочный файл будет включён более одного раза, вы получите несколько определений, что приведёт к ошибке компоновщика).

Для шаблонных классов определение (шаблонного) статического элемента обычно размещается непосредственно под определением шаблонного класса в заголовочном файле (это не нарушает принцип единой ответственности, поскольку такие определения неявно встраиваются).

Инициализация статических переменных-элементов внутри определения класса

Есть несколько способов упростить описанный выше процесс. Во-первых, если статический элемент является константным целочисленным типом (включая `char` и `bool`) или константным перечислением, статический элемент можно инициализировать внутри определения класса:

```
class Whatever
{
public:
    static const int s_value{ 4 }; // a static const int can be defined and initialized directly
};
```

В приведённом выше примере, поскольку статическая переменная-элемент является константой типа `int`, строка явного определения не требуется. Этот приём разрешён, поскольку эти конкретные типы констант являются константами времени компиляции.

Статические элементы могут быть встроенными переменными:

```
class Whatever
{
public:
    static inline int s_value{ 4 }; // a static inline variable can be defined and initialized directly
};
```

Такие переменные могут быть инициализированы внутри определения класса независимо от того, являются ли они константами. Это предпочтительный способ определения и инициализации статических элементов.

Поскольку элементы `constexpr` являются неявными встроенными (начиная с C++17), статические элементы `constexpr` также могут быть инициализированы внутри определения класса без явного использования ключевого слова `inline`:

```
#include <string_view>

class Whatever
{
public:
    static constexpr double s_value{ 2.2 }; // ok
    static constexpr std::string_view s_view{ "Hello" }; // this even works for classes that support constexpr initialization
};
```

Наилучшая практика

Сделайте статические элементы `inline` или `constexpr` так, чтобы их можно было инициализировать внутри определения класса.

Пример статических переменных- элементов

Зачем использовать статические переменные внутри классов? Одна из целей — присвоить уникальный идентификатор каждому экземпляру класса. Вот пример:

```
#include <iostream>

class Something
{
private:
    static inline int s_idGenerator { 1 };
    int m_id {};

public:
    // grab the next value from the id generator
    Something() : m_id { s_idGenerator++ }
```

```

    {
    }

    int getID() const { return m_id; }
};

int main()
{
    Something first{};
    Something second{};
    Something third{};

    std::cout << first.getID() << '\n';
    std::cout << second.getID() << '\n';
    std::cout << third.getID() << '\n';
    return 0;
}

```

Эта программа печатает:

```

1
2
3

```

Поскольку `s_idGenerator` является общим для всех объектов `Something`, при создании нового объекта `Something` конструктор инициализирует `m_id` текущим значением `s_idGenerator` и затем увеличивает значение для следующего объекта. Это гарантирует, что каждый созданный объект `Something` получает уникальный идентификатор (увеличивающийся в порядке создания).

Присвоение каждому объекту уникального идентификатора может помочь при отладке, так как его можно использовать для различения объектов с идентичными данными. Это особенно актуально при работе с массивами данных.

Статические переменные-элементы также полезны, когда классу необходимо использовать таблицу поиска (например, массив, используемый для хранения набора предварительно рассчитанных значений). Если сделать таблицу поиска статической, то для всех объектов будет существовать только одна копия, а не по одной копии для каждого созданного объекта. Это может значительно сэкономить память.

Статические функции-элементы

Статические переменные-элементы — это переменные-элементы, которые принадлежат классу, а не объектам класса. Если статическая переменная-элемент является общедоступной, к ней можно получить прямой доступ, используя имя класса и оператор разрешения области видимости:

```
#include <iostream>

class Something
{
public:
    static inline int s_value { 1 };
};

int main()
{
    std::cout << Something::s_value; // s_value is public, we can access it directly
}
```

Но что, если статическая переменная-элемент является закрытой? Рассмотрим следующий пример:

```
#include <iostream>

class Something
{
private: // now private
    static inline int s_value { 1 };
};

int main()
{
    std::cout << Something::s_value; // error: s_value is private and can't be accessed directly outside the class
}
```

В этом случае мы не можем получить доступ к `Something::s_value` напрямую из `main()`, потому что он является закрытым. Обычно мы получаем доступ к закрытым элементам через открытые функции-элементы. Хотя мы могли бы создать обычную открытую функцию-элемент для доступа к `s_value`, нам пришлось бы создавать экземпляр объекта типа класса, чтобы использовать функцию!

```
#include <iostream>

class Something
{
private:
    static inline int s_value { 1 };

public:
    int getValue() { return s_value; }
};

int main()
{
    Something s{};
    std::cout << s.getValue(); // works, but requires us to instantiate an object to call getValue()
}
```

Мы можем добиться большего.

Статические функции-элементы

Переменные-элементы — не единственный тип элементов, которые можно сделать статическими. Функции-элементы также можно сделать статическими. Вот приведённый выше пример со статической функцией-элементом:

```
#include <iostream>

class Something
{
private:
    static inline int s_value { 1 };

public:
    static int getValue() { return s_value; } // static member function
};

int main()
{
    std::cout << Something::getValue() << '\n';
}
```

Поскольку статические функции-элементы не связаны с конкретным объектом, их можно вызывать напрямую, используя имя класса и оператор разрешения области видимости (например, `Something::getValue()`). Как и статические переменные-элементы, их также можно вызывать через объекты типа класса, хотя это не рекомендуется.

Статические функции-элементы не имеют `this` указателя

У статических функций-элементов есть две интересные особенности, на которые стоит обратить внимание. Во-первых, поскольку статические функции-элементы не привязаны к объекту, у них нет `this` указателя! Это имеет смысл, если подумать: `this` указатель всегда указывает на объект, с которым работает функция-элемент. Статические функции-элементы не работают с объектом, поэтому `this` указатель не нужен.

Во-вторых, статические функции-элементы могут напрямую обращаться к другим статическим элементам (переменным или функциям), но не к нестатическим. Это связано с тем, что нестатические элементы должны принадлежать объекту класса, а у статических функций-элементов нет объекта класса для работы!

Статические элементы, определенные вне определения класса

Статические функции-элементы также могут быть определены вне объявления класса. Это работает так же, как и для обычных функций-элементов.

```
#include <iostream>

class IDGenerator
{
private:
    static inline int s_nextID { 1 };

public:
    static int getNextID(); // Here's the declaration for a static function
};

// Here's the definition of the static function outside of the class. Note we don't
// use the static keyword here.
int IDGenerator::getNextID() { return s_nextID++; }

int main()
{
    for (int count{ 0 }; count < 5; ++count)
        std::cout << "The next ID is: " << IDGenerator::getNextID() << '\n';

    return 0;
}
```

```
}
```

Эта программа печатает:

```
Следующий идентификатор: 1
Следующий идентификатор: 2
Следующий идентификатор: 3
Следующий идентификатор: 4
Следующий идентификатор: 5
```

Обратите внимание, что, поскольку все данные и функции в этом классе являются статическими, нам не нужно создавать экземпляр объекта класса, чтобы использовать его функции! Этот класс использует статическую переменную-элемент для хранения значения следующего идентификатора, который будет присвоен, и предоставляет статическую функцию-элемент для возврата этого идентификатора и его увеличения.

Классы и заголовочные файлы, функции-элементы, определённые внутри определения класса, неявно встраиваются. Функции-элементы, определённые вне определения класса, неявно не встраиваются, но могут быть встроены с помощью ключевого слова `inline`. Поэтому статическая функция-элемент, определённая в заголовочном файле, должна быть встроена с помощью `inline`, чтобы не нарушать правило единого определения (ODR), если этот заголовочный файл затем включается в несколько единиц трансляции.

Небольшое предупреждение о классах со всеми статическими элементами

Будьте осторожны при написании классов со всеми статическими элементами. Хотя такие «чистые статические классы» (также называемые «моностатическими») могут быть полезны, у них есть и потенциальные недостатки.

Во-первых, поскольку все статические элементы создаются только один раз, невозможно создать несколько копий чистого статического класса (без клонирования класса и его переименования). Например, если вам нужны две независимые `IDGenerator`, это невозможно сделать с помощью чистого статического класса.

Во-вторых, на уроке о глобальных переменных вы узнали, что глобальные переменные опасны, потому что любой фрагмент кода может изменить значение глобальной переменной и в итоге нарушить работу другого фрагмента кода, который, казалось бы, не имеет к нему отношения. То же самое относится и к чистым статическим классам. Поскольку все элементы принадлежат классу (а не объекту класса), а объявления классов обычно имеют глобальную область видимости, чистый статический класс, по сути, эквивалентен объявлению функций и глобальных переменных в глобально доступном пространстве имён со всеми вытекающими недостатками глобальных переменных.

Вместо того, чтобы писать класс со всеми статическими элементами, рассмотрите возможность написания обычного класса и создания его глобального экземпляра (глобальные переменные имеют статическую продолжительность существования). Таким образом, глобальный экземпляр можно использовать при необходимости, но локальные экземпляры всё равно можно создавать, если это полезно.

Чистые статические классы против пространств имён

Чистые статические классы во многом схожи с пространствами имён. И те, и другие позволяют определять переменные со статической продолжительностью жизни и функции в пределах своей области видимости. Однако одно существенное различие заключается в том, что классы имеют контроль доступа, а пространства имён — нет.

В целом, статический класс предпочтительнее, если у вас есть статические элементы данных и/или вам нужен контроль доступа. В противном случае лучше использовать пространство имён.

C++ не поддерживает статические конструкторы

Если вы можете инициализировать обычные переменные-элементы с помощью конструктора, то логично, что вы можете инициализировать статические переменные-элементы с помощью статического конструктора. И хотя некоторые современные языки поддерживают статические конструкторы именно для этой цели, C++, к сожалению, не входит в их число.

Если ваша статическая переменная может быть инициализирована напрямую, конструктор не нужен: вы можете инициализировать статическую переменную-элемент в точке определения (даже если она является закрытой). Мы делаем это в примере IDGenerator выше. Вот ещё один пример:

```
#include <iostream>

struct Chars
{
    char first{};
    char second{};
    char third{};
    char fourth{};
    char fifth{};
};

struct MyClass
{
    static inline Chars s_mychars { 'a', 'e', 'i', 'o', 'u' }; // initialize static
variable at point of definition
};

int main()
{
    std::cout << MyClass::s_mychars.third; // print i

    return 0;
}
```

Если для инициализации статической переменной-элемента требуется выполнение кода (например, цикла), существует множество различных, несколько запутанных способов сделать это. Один из способов, который работает со всеми переменными, статическими или нет, — использовать функцию для создания объекта, заполнения его данными и возврата вызывающей стороне. Это возвращаемое значение можно скопировать в инициализируемый объект.

```
#include <iostream>

struct Chars
{
    char first{};
    char second{};
    char third{};
    char fourth{};
    char fifth{};
};

class MyClass
{
private:
    static Chars generate()
    {
        Chars c{}; // create an object
        c.first = 'a'; // fill it with values however you like
        c.second = 'e';
        c.third = 'i';
        c.fourth = 'o';
        c.fifth = 'u';

        return c; // return the object
    }

public:
```

```
static inline Chars s_mychars { generate() }; // copy the returned object into
s_mychars
};

int main()
{
    std::cout << MyClass::s_mychars.third; // print i

    return 0;
}
```

Связанный контент

Для этого также можно использовать лямбду.